Layer 1 Tower Photo Classification: Concepts for Beginners

1. The problem

A drone photographs a transmission tower. We want software to sort each photo into a category (**Phase, Top, Earth_wire, Bottom**).

We only have a few hundred labeled photos, and some classes have just 13 to 26. That is far too few to train a deep neural network from scratch, because the network would memorize the photos instead of learning general patterns (this is called overfitting).

2. The solution: transfer learning

We reuse a network that someone else already trained on millions of everyday images (ImageNet), and train only a very small classifier on top.

1. **Frozen ResNet18** turns each photo into a list of 512 numbers (a *feature vector*). We never change this network.
2. **Logistic regression** learns to map those 512 numbers to a class. This is the only part we train.

```
Photo -> resize/crop/normalize -> frozen ResNet18 -> 512 numbers -> logistic regression -> class
```

Why this works with little data: the heavy visual learning is already done. The small classifier has few things to learn, so it overfits much less.

3. What Layer 1 does

Layer 1 looks at the **whole photo** and decides which of four classes it belongs to. It does not look for small parts inside the photo.

Class	Meaning
Phase	Photos of the phase conductors and their insulators (circuit 1 and circuit 2, all of R/Y/B merged)
Top	Photos of the top of the tower (tower centre)
Earth_wire	Photos of the earth wire
Bottom	Photos of the tower bottom (no data collected yet)

Method: frozen ResNet18 for features, plus logistic regression as the classifier.

4. The pipeline we built

Step	Script	What it does
1	`reorganize_tower_folders.py`	Converts labeled circuit and earthwire folders into Phase / Top / Earth_wire / Bottom per tower
2	`build_training_dataset.py`	Pools all towers into one folder per class and checks for empty or damaged files
3	`build_xy.py`	Extracts features (with optional augmentation) and builds **X** (features) and **y** (labels)

4	`train_classifier.py`	Trains logistic regression and reports cross-validation results
5	`classify.py`	Predicts classes for new, unlabeled towers and records a confidence score
6	`sort.py`	Rebuilds the per-tower folder structure from the predictions

Safety habits used in every script: the input folder is only read, never changed; every script has a `--dry_run` mode; files are copied, not moved; re-running is safe.

5. Key ideas explained simply

Feature vector. A photo is turned into 512 numbers describing which visual patterns appear in it. Similar photos give similar numbers.

Augmentation. We make slightly changed copies of a photo (flip, small rotation, brightness change). This gives the classifier more varied examples without taking new photos.

Cross-validation (CV). To estimate how well the model works on new data, we split the data into 5 parts (*folds*). We train on 4 and test on the 5th, rotate five times, and average the results. *Stratified* means every fold keeps the same class proportions.

Confidence. The classifier gives a probability for every class. The highest probability is the prediction, and its value is the confidence. Low confidence means the photo should be checked by a person.

6. Measuring accuracy: the metrics

Start with the **confusion matrix**. Rows are the true class and columns are the predicted class. The diagonal shows correct predictions, and everything else shows which classes get mixed up. Every metric below comes from it.

Metric	Question it answers	Watch out for
Accuracy	What fraction of all predictions were right?	Misleading when one class dominates. Always saying “Phase” can still score about 91%.
Precision (per class)	Of the photos called “Top”, how many really were Top?	Measures false alarms
Recall (per class)	Of the real Top photos, how many did we find?	Measures misses
F1	One number combining precision and recall	Good for comparing classes
Macro average	Average across classes, each class counts equally	The honest number for small classes
Weighted average	Average weighted by class size	Dominated by the biggest class, can hide problems
AP (Average Precision)	How well does the confidence score rank true photos above false ones, for one class?	Uses probabilities, not just labels
mAP	AP averaged over all classes	Rewards confident-when-right, unsure-when-wrong models

Computing mAP:

```
        from sklearn.metrics import average_precision_score
        from sklearn.preprocessing import label_binarize
```

```
        probs = clf.predict_proba(X_test)
        y_bin = label_binarize(y_test, classes=range(n_classes))
        ap = [average_precision_score(y_bin[:, k], probs[:, k]) for k in range(n_classes)]
        mAP = sum(ap) / len(ap)
```

7. Reading our current results honestly

Cross-validation gave Phase 1.00 / 1.00, Earth_wire recall 0.95, Top precision 0.91, and macro F1 around 0.97. These look strong, but be careful:

- **Very small test counts.** Top had about 10 test images and Earth_wire about 20. One photo changes the score noticeably.
- **Probably optimistic.** Drone frames taken seconds apart look almost identical. If near-duplicates land in both train and test, the test is too easy. Augmented copies of the same photo cause the same problem.
- **The real test is new towers.** Keep whole towers aside that the model has never seen.

8. How to improve accuracy

Measure better first (most important)

1. Split by tower, not by photo. Use `GroupKFold` with the tower name as the group.
2. Keep augmented copies of a photo in the same fold as the original.
3. Report macro F1, per-class AP, mAP and the confusion matrix on held-out towers.
4. Look at the actual wrong photos. Error analysis often teaches more than any single number.

Improve the data

5. Collect more Top and Earth_wire photos, the weakest and rarest classes.
6. Add photos in different lighting, weather and angles.
7. Fix labeling mistakes and remove empty or corrupted files.

Improve the model (change one thing at a time)

8. Tune the regularization strength `C` (try 0.01, 0.1, 1, 10) with tower-grouped cross-validation.
9. Try a stronger frozen backbone (ResNet50, or a self-supervised model such as DINOv2).
10. Use test-time augmentation: average predictions over a few flipped or cropped copies.
11. Fine-tune the last ResNet block only later, and only with enough data. With about 100 photos it tends to overfit.

Improve the workflow

12. Use `--confidence_threshold` so uncertain photos go to an `Uncertain/` folder for human review. Reviewed photos become new training data.
13. Calibrate the probabilities if the confidence scores look unreliable.

9. Suggested next steps

1. Re-run evaluation with tower-grouped cross-validation and record macro F1 and mAP as the baseline.
2. Fix the filename mismatch: `train_classifier.py` loads `step4_X.npy`, but `build_xy.py` now saves `build_X.npy`.
3. Collect more Top and Earth_wire photos.